

Modul mahasiswa Lab 05 - Dockerfile dan Cloud Notes API

Kebijakan kelas: Lab ini latihan formatif, tanpa tugas, nilai, atau penyerahan terpisah. Satu proyek besar dikerjakan oleh kelompok **3 orang**, dengan presentasi checkpoint minggu 7 (UTS) dan hasil akhir minggu 14 (UAS). Simpan hasil lab hanya bila berguna sebagai referensi atau bukti proses proyek. Baca [brief proyek kelompok](#). Bobot resmi tetap mengikuti RPS/LMS.

Sesi RPS: 05 · **Jalur utama:** build dan run lokal · **Lanjutkan ke:** repo Lab 06 setelah praktik ini

Hasil belajar

Anda dapat menjelaskan tiap tahap [Dockerfile](#), membangun image bertag, menguji REST API dalam container, menunjukkan proses berjalan sebagai user non-root, dan menjelaskan mengapa catatan hilang setelah container diganti. Kode FastAPI ada di [app/main.py](#); dependensi ada di [requirements.txt](#).

Teori singkat. Dockerfile adalah resep build image. Tahap [builder](#) memasang dependensi Python ke virtual environment; tahap [runtime](#) menyalin hanya hasil yang diperlukan serta kode aplikasi. [.dockerignore](#) mengurangi berkas yang masuk ke build context. [RUN](#) dieksekusi saat build; [CMD](#) memberi perintah default saat container mulai, sedangkan [ENTRYPOINT](#) dapat menetapkan executable utama. [ENV](#) pada Dockerfile berbeda dari [-e](#) pada [docker run](#): [-e](#) memasok nilai saat container dijalankan. Data API ini disimpan di memori proses, belum di volume/database.

Pilihan demo kelas: [panduan online](#) menjalankan API utama hanya dengan browser; [panduan Compose](#) menambahkan frontend dan PostgreSQL sebagai contoh terpisah. Keduanya latihan formatif. API utama memakai port host 8000 dan memori; default Compose memakai API 13005, frontend 13006, serta database internal [db:5432](#). Screenshot Compose online memakai alternatif API **13105** dan frontend **13106** untuk menghindari proses lain pada host 13005; panduan menunjukkan perubahan [.env](#) dan pemilihan port yang benar. Port di dalam container tetap API 8000/web 80. Jangan menukar URL atau menganggap hasil memori pada contoh pertama sudah persisten.

Persiapan dan build

Pastikan Docker Engine berjalan, port host **8000** kosong, dan terminal berada di root repo Lab 05. Di PowerShell, Bash, atau WSL:

Gambar 1. Versi Docker dan instruksi penting dalam Dockerfile Lab 05

```
docker version
docker build -t cloud-notes-api:lab05 .
docker image ls cloud-notes-api
docker run --name cloudlab-api -d -p 127.0.0.1:8000:8000 -e APP_ENV=classroom cloud-notes-api:lab05
docker ps --filter name=cloudlab-api
docker logs cloudlab-api
docker exec cloudlab-api id
docker image inspect cloud-notes-api:lab05 --format '{{.Size}} bytes; user={{.Config.User}}'
```

Lab 05 · baca Dockerfile sebelum build

```
$ docker version --format '{{.Server.Version}}'
```

```
29.6.2
```

```
$ Select-String -Path Dockerfile -Pattern '^(FROM|RUN|COPY|USER|EXPOSE|CMD)'
```

```
Dockerfile:1:FROM python:3.12-slim AS builder
Dockerfile:3:COPY requirements.txt .
Dockerfile:4:RUN python -m venv /opt/venv && /opt/venv/bin/pip install --no-cache-dir -r requirements.txt
Dockerfile:6:FROM python:3.12-slim AS runtime
Dockerfile:10:RUN useradd --create-home --uid 10001 appuser
Dockerfile:12:COPY --from=builder /opt/venv /opt/venv
Dockerfile:13:COPY app ./app
Dockerfile:14:USER appuser
Dockerfile:15:EXPOSE 8000
Dockerfile:16:CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Output aktual: FROM membentuk tahap image; RUN saat build; USER/CMD dipakai saat container dimulai.

Perintah: `docker version --format '{{.Server.Version}}'` dan PowerShell `Select-String -Path Dockerfile -Pattern '^(FROM|RUN|COPY|USER|EXPOSE|CMD)'`.

Fungsi: memastikan Engine aktif dan membaca tahap build/runtime sebelum build. Cara kerja: pencarian memilih instruksi Dockerfile; **FROM** membuat tahap, **RUN** memasang dependensi, **USER** memilih proses non-root, **CMD** menjalankan API. Baca hasil: dua **FROM**, **USER** `appuser`, dan port 8000. Ini render output command aktual.

Checkpoint A: image `cloud-notes-api:lab05` terdaftar; log menunjukkan Uvicorn menerima koneksi pada port 8000. Tunggu beberapa detik bila API belum siap segera setelah `docker run`. Buka <http://127.0.0.1:8000/docs>. Dokumentasi OpenAPI yang Anda lihat akan mirip screenshot hasil run lokal berikut.

Gambar 2. Tag image, user runtime non-root, dan pemetaan port dari Docker

```
docker build -t cloud-notes-api:lab05 .  
docker exec cloudlab-api id  
docker image inspect
```

Cloud Services - hasil perintah lokal

Image build dan user runtime

Output aktual dari sesi uji; ditata ulang agar mudah dibaca pada modul.

```
docker image inspect; docker exec id; docker ps
```

```
cloud-notes-api:lab05 | user=appuser | 52164335 bytes
```

```
docker exec cloudlab-api id
```

```
uid=10001(appuser) gid=10001(appuser) groups=10001(appuser)
```

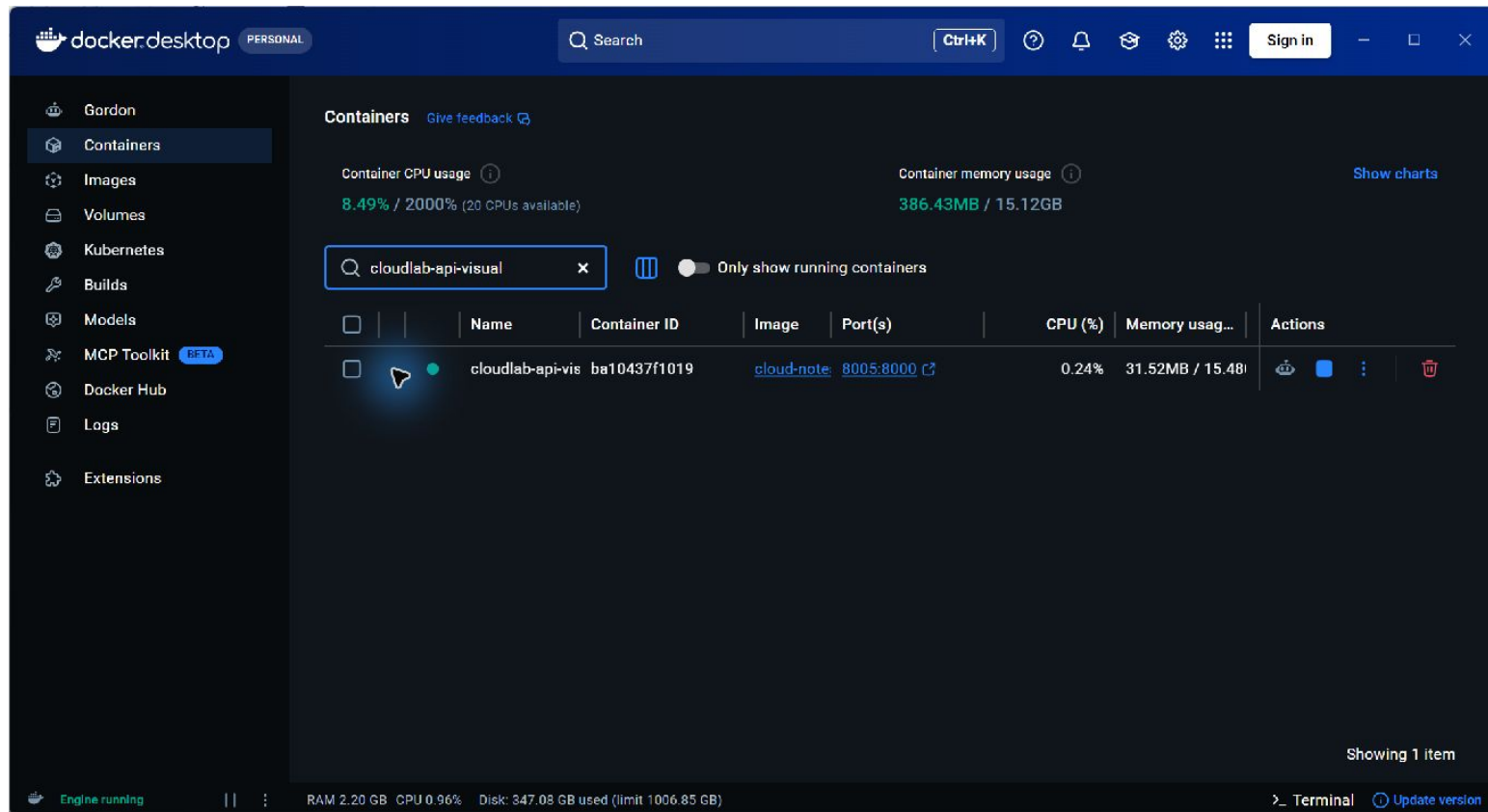
NAMES	IMAGE	PORTS
cloudlab-api	cloud-notes-api:lab05	127.0.0.1:8000->8000/tcp

Langkah: Jalankan `docker build -t cloud-notes-api:lab05 .`, `docker exec cloudlab-api id`, dan `docker image inspect`. Fungsi: Membuktikan build API, user runtime non-root, serta port yang dipakai. Cara kerja: Dockerfile multi-stage membuat image; proses API berjalan sebagai appuser di dalam container. Baca hasil: Baca tag image, UID 10001/appuser, dan mapping host 8000 ke container 8000; ukuran image dapat berbeda.

Gambar: hasil aktual `docker image inspect`, `docker exec id`, dan `docker ps`. Pada mesin Anda, periksa `appuser`, `UID 10001`, dan `port host 8000`; ukuran image dapat berbeda.

Gambar 3. Docker Desktop menampilkan container API Lab 05 yang berjalan

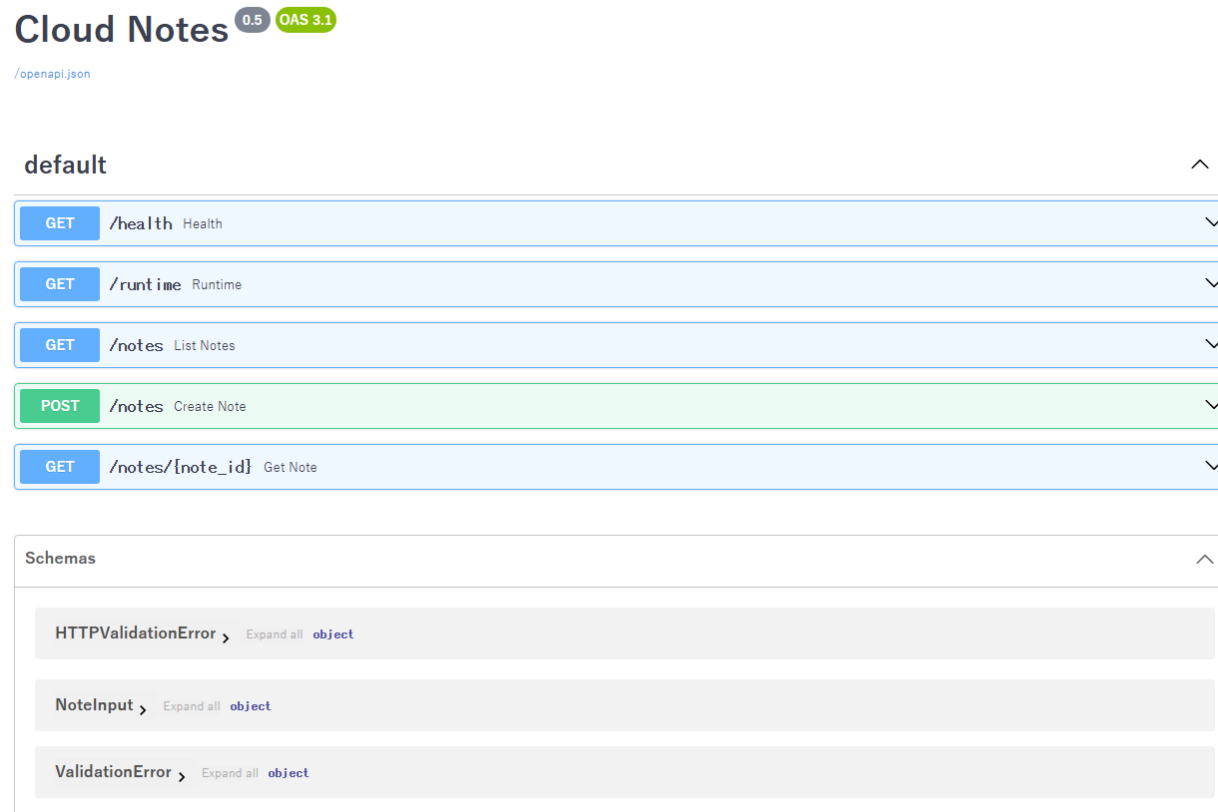
```
docker run
docker ps
```



Langkah: Buka Docker Desktop > Containers setelah `docker run`, lalu bandingkan dengan `docker ps`. Fungsi: Melihat status container dan pemetaan port tanpa kehilangan arti perintah CLI. Cara kerja: Docker menjalankan image `cloud-notes-api:lab05` sebagai container; titik hijau menandai proses sedang berjalan. Baca hasil: Pada screenshot uji, host 8005 diteruskan ke port container 8000 (`docker run --name cloudlab-api-visual -d -p 127.0.0.1:8005:8000 ...`) karena Lab 06 memakai host 8000 pada waktu yang sama. Langkah utama Anda tetap memakai `127.0.0.1:8000:8000` dan URL port 8000; status HTTP 200 diuji terpisah agar titik hijau tidak disalahartikan sebagai healthcheck.

Gambar 4. OpenAPI Cloud Notes dari image Lab 05 yang berjalan

```
docker run -p 127.0.0.1:8000:8000  
http://127.0.0.1:8000/docs
```



Langkah: Setelah `docker run -p 127.0.0.1:8000:8000`, buka `http://127.0.0.1:8000/docs`. Fungsi: Mengenali kontrak endpoint API Cloud Notes. Cara kerja: FastAPI membangun Swagger UI dari skema OpenAPI aplikasi yang hidup di container. Baca hasil: Cari route `/health`, `/runtime`, dan `/notes`; buka satu route untuk melihat metode serta bentuk respons.

Praktik API dan validasi

Di PowerShell:

```
Invoke-RestMethod http://127.0.0.1:8000/health
Invoke-RestMethod http://127.0.0.1:8000/runtime
Invoke-RestMethod http://127.0.0.1:8000/notes -Method Post -ContentType application/json -Body `
'{"title":"Docker","content":"Catatan kelas"}'
Invoke-RestMethod http://127.0.0.1:8000/notes
try { Invoke-WebRequest http://127.0.0.1:8000/notes -Method Post -ContentType application/json -Body `
'{"title":"","content":"uji"}' -UseBasicParsing } catch { [int]$_Exception.Response.StatusCode }
```

Di Bash/WSL:

```
curl -fsS http://127.0.0.1:8000/health
curl -fsS http://127.0.0.1:8000/runtime
curl -fsS -X POST http://127.0.0.1:8000/notes -H 'Content-Type: application/json' -d \
'{"title":"Docker","content":"Catatan kelas"}'
curl -fsS http://127.0.0.1:8000/notes
curl -s -o /dev/null -w '%{http_code}\n' -X POST http://127.0.0.1:8000/notes -H \
'Content-Type: application/json' -d '{"title":"","content":"uji"}'
```

Checkpoint B: `/health` bernilai `ok`, `/runtime` menampilkan `APP_ENV=classroom` dan `storage=memory`, POST membuat catatan, GET menampilkannya, dan judul kosong menghasilkan HTTP **422**. ID dan urutan catatan bisa berbeda. Screenshot berikut adalah contoh respons dari container lokal; buat bukti sendiri saat praktik.

Gambar 5. Respons aktual health, runtime, POST, dan GET Lab 05

```
/health  
/runtime  
/notes  
/notes  
curl  
Invoke-RestMethod
```



```
Lab 05: respons API lokal  
  
Invoke-RestMethod /health, /runtime, POST /notes, GET /notes  
  
GET /health  
{ "status": "ok", "service": "cloud-notes-api" }  
  
GET /runtime  
{ "environment": "classroom", "storage": "memory" }  
  
POST /notes  
{ "id": 1, "title": "Docker", "content": "Catatan kelas" }  
  
GET /notes  
{ "value": [{ "id": 1, "title": "Docker", "content": "Catatan kelas" } ], "Count": 1 }
```

Langkah: Kirim GET `/health`, GET `/runtime`, POST `/notes`, dan GET `/notes` dengan `curl` atau `Invoke-RestMethod`. Fungsi: Menguji kesehatan, identitas runtime, dan alur tulis/baca catatan. Cara kerja: Request host masuk melalui port publish ke FastAPI; POST menyimpan catatan dalam memori proses. Baca hasil: Baca `status` sehat, `appuser`, respons catatan baru, lalu item yang sama pada daftar GET.

Hapus container, jalankan image yang sama lagi, lalu GET daftar catatan. Gunakan perintah sesuai terminal:

```
docker rm -f cloudlab-api
docker run --name cloudlab-api -d -p 127.0.0.1:8000:8000 -e APP_ENV=classroom cloud-notes-api:lab05
```

PowerShell: `Invoke-RestMethod http://127.0.0.1:8000/notes`; Bash: `curl -fsS`

`http://127.0.0.1:8000/notes`. **Checkpoint C:** daftar dimulai kosong setelah proses/container baru siap. Jelaskan penyebabnya dan bandingkan dengan Lab 06 nanti.

Gambar 6. HTTP 422 untuk input kosong dan daftar kosong sesudah container dibuat ulang

```
/notes  
docker rm -f cloudlab-api
```

Cloud Services - hasil perintah lokal

Validasi dan reset data memori

Output aktual dari sesi uji, ditata ulang agar mudah dibaca pada modul.

POST /notes; input invalid; docker rm/run; GET /notes

POST /notes sebelum container diganti

```
{"id":1,"title":"Sementara","content":"Hilang saat container diganti"}
```

POST /notes dengan title kosong

HTTP 422

GET /notes setelah docker rm + docker run

```
[]
```

Langkah: POST judul kosong ke /notes, lalu `docker rm -f cloudlab-api` dan jalankan container baru dari image sama. Fungsi: Mendemonstrasikan validasi input dan batas penyimpanan in-memory. Cara kerja: FastAPI menolak payload tidak valid; container baru memulai proses dengan memori kosong. Baca hasil: Cari HTTP 422 dan daftar catatan kosong setelah restart lewat container baru.

Gambar: output aktual dari POST valid, POST berjudul kosong, dan GET setelah `docker rm/docker run`. ID serta isi catatan Anda boleh berbeda.

Tantangan kode mandiri: tambahkan endpoint `GET /stats` pada salinan Anda di `app/main.py`. Kembalikan `note_count` dan `longest_title_chars`; saat belum ada catatan keduanya bernilai 0. Gunakan lock yang sama dengan operasi catatan, uji sebelum dan sesudah membuat dua catatan, lalu build ulang image dengan tag berbeda. Tantangan ini latihan tambahan; endpoint inti di atas harus berhasil lebih dahulu.

Diskusi, catatan opsional, dan Git

Diskusikan bersama dosen: (1) bagian mana dari Dockerfile berjalan saat build dan bagian mana saat run; apa beda `CMD` dan `ENTRYPOINT` dalam konteks ini; (2) mengapa `.dockerignore` memuat `.env`; (3) apa arti port kiri/kanan pada `127.0.0.1:8000:8000`; (4) apa hubungan validasi `Field(min_length=1)` dengan HTTP 422; (5) apa kelemahan menyimpan catatan hanya di memori.

Pertanyaan tersebut dipakai untuk diskusi kelas, bukan penyerahan atau penilaian terpisah; kuncinya tersedia pada bagian akhir. Catatan dan Git di bawah bersifat opsional sebagai referensi proses proyek kelompok.

Jika ingin menyimpan catatan proses proyek, dari root repo salin `template laporan` menjadi `hasil/lab05.md`: PowerShell `Copy-Item hasil/TEMPLATE_LAPORAN.md hasil/lab05.md`; Bash `cp hasil/TEMPLATE_LAPORAN.md hasil/lab05.md`. Catat hasil nyata, perintah, jawaban, dan screenshot Anda di `hasil/bukti/`. Catatan ini opsional; jangan memakai screenshot modul sebagai bukti pribadi.

```
docker rm -f cloudlab-api
```

Dari root repo Lab 05, ikuti [panduan Git](#) untuk repo sendiri:

```
git status --short
git add Dockerfile .dockerignore requirements.txt app MODUL_MAHASISWA.md hasil/lab05.md hasil/bukti
git diff --cached --name-only
git diff --cached --check
git commit -m "lab05: build dan uji Cloud Notes API"
git push
```

Gambar 7. Repo template Lab 05 terbit dan commit lokal sama dengan GitHub

```
git remote -v
git status --short
git log -1 --oneline
git rev-parse HEAD
git ls-remote origin refs/heads/main
```

Lab 05 · cek repo Git terbit

```
$ git remote -v
origin https://github.com/SeedFlora/meet5CloudService.git (fetch)
origin https://github.com/SeedFlora/meet5CloudService.git (push)
```

```
$ git status --short
?? screenshots/lab05_dockerfile_persiapan.png
```

```
$ git log -1 --oneline
ace4300 Publish complete Lab 05 Dockerfile modules
```

```
$ git rev-parse HEAD; git ls-remote origin refs/heads/main
local: ace4300f75c623dbb3aa8bb2528350484170f48f
remote: ace4300f75c623dbb3aa8bb2528350484170f48f
Sama: True
```

Output aktual repo template SeedFlora. Mahasiswa melakukan commit/push pada repo pribadi.

SHA pada gambar adalah snapshot saat uji. Setelah modul diperbarui, jalankan ulang perintah untuk memeriksa commit terbaru. Perintah: `git remote -v`, `git status --short`, `git log -1 --oneline`, `git rev-parse HEAD`, dan `git ls-remote origin refs/heads/main`. Fungsi: memeriksa tujuan serta hasil push kode. Cara kerja: Git membandingkan SHA commit lokal dengan SHA main remote. Baca hasil: Sama: True untuk repo pengajar; mahasiswa mengulanginya pada repo pribadi. Ini render output command aktual.

Jika belum ada folder bukti, hilangkan argumen itu dari `git add`. Pastikan staged tidak berisi `.env`, kredensial, atau hasil build. Jika push pertama belum punya upstream, jalankan `git push -u origin main`.

Jika gagal: konflik port 8000 biasanya berarti container Lab 06 atau Lab 05 lama masih aktif (`docker ps`). Jika `docker build` gagal mengambil image/dependensi, cek koneksi internet dan jalankan ulang. Jika aplikasi belum siap setelah `run`, tunggu status log Uvicorn sebelum memanggil API.

Pilihan cloud - Docker Hub: bila dosen meminta publish image dan Anda punya akun, jalankan `docker tag cloud-notes-api:lab05 NAMA_AKUN/cloud-notes-api:lab05`, `docker login`, lalu `docker push NAMA_AKUN/cloud-notes-api:lab05`. Ganti `NAMA_AKUN`; jangan merekam token login atau memasukkannya ke repo. Push Git untuk kode tetap terpisah dari push image ke registry.

Kunci tantangan dan kasus kerja harian

Kasus: tim support perlu melihat jumlah tiket/catatan serta judul terpanjang tanpa membuka semua isi catatan. Anda menambahkan endpoint statistik baca saja. Kunci ada di bawah; lakukan percobaan sendiri dahulu, lalu bandingkan hasil. Semua perintah dijalankan dari **root repo Lab 05**.

1. Baca kontrak aplikasi

Dockerfile tahap `builder` memasang paket ke `/opt/venv`; tahap `runtime` menyalin hasilnya, memasang kode, dan menjalankan Uvicorn sebagai UID 10001. `docker build` menjalankan instruksi build dan menghasilkan image. `docker run` membuat proses container dari image tersebut. Port kiri `127.0.0.1:8000` adalah port laptop yang hanya menerima koneksi lokal; port kanan `8000` milik proses dalam container.

2. Kunci kode GET /stats

Tambahkan potongan berikut ke akhir `app/main.py`, di luar fungsi lain. Variabel `_notes` dan `_lock` sudah disediakan oleh starter; jangan membuat dictionary atau lock baru karena hasilnya akan terpisah dari endpoint `/notes`.

```
@app.get("/stats")
def stats() -> dict[str, int]:
    with _lock:
        titles = [item["title"] for item in _notes.values()]
        return {
            "note_count": len(titles),
            "longest_title_chars": max((len(title) for title in titles), default=0),
        }
```

Lock memastikan snapshot daftar judul konsisten bila ada request POST bersamaan. `default=0` membuat hasil awal tetap valid ketika belum ada catatan. Hitung panjang judul yang sudah divalidasi dan dipangkas spasinya oleh `NoteInput`.

3. Build ulang dan uji dari awal sampai akhir

Gunakan tag baru supaya jelas image mana yang berisi solusi. Hapus container percobaan Lab 05 yang lama bila masih ada, lalu:

```
docker build -t cloud-notes-api:stats .
docker rm -f clouddlab-api
docker run --name clouddlab-api -d -p 127.0.0.1:8000:8000 -e APP_ENV=classroom cloud-notes-api:stats
```

Jika `docker rm` melaporkan container tidak ada, lanjutkan ke `docker run`. Tunggu `/health` menjawab HTTP 200. Di **PowerShell**, jalankan:

```
Invoke-RestMethod http://127.0.0.1:8000/stats
Invoke-RestMethod http://127.0.0.1:8000/notes -Method Post -ContentType application/json -Body `
'{"title":"Ops","content":"Pertama"}'
Invoke-RestMethod http://127.0.0.1:8000/notes -Method Post -ContentType application/json -Body `
'{"title":"Incident 42","content":"Kedua"}'
Invoke-RestMethod http://127.0.0.1:8000/stats
try { Invoke-WebRequest http://127.0.0.1:8000/notes -Method Post -ContentType application/json -Body `
'{"title":"    ","content":"uji"}' -UseBasicParsing } catch { [int]$_._Exception.Response.StatusCode }
```

Di **Bash/Git Bash**, padanannya:

```
curl -fsS http://127.0.0.1:8000/stats
curl -fsS -X POST http://127.0.0.1:8000/notes -H 'Content-Type: application/json' -d \
'{"title":"Ops","content":"Pertama"}'
curl -fsS -X POST http://127.0.0.1:8000/notes -H 'Content-Type: application/json' -d \
'{"title":"Incident 42","content":"Kedua"}'
curl -fsS http://127.0.0.1:8000/stats
curl -s -o /dev/null -w '%{http_code}\n' -X POST http://127.0.0.1:8000/notes -H \
'Content-Type: application/json' -d '{"title":"    ","content":"uji"}'
```

Hasil kunci: awal `{"note_count":0,"longest_title_chars":0}`; sesudah dua POST `{"note_count":2,"longest_title_chars":11}`; judul spasi saja ditolak HTTP **422**. Jika hasil `/stats` masih 404, image baru belum dibuild atau container masih memakai tag lama. Cek dengan `docker ps --filter name=cloudlab-api --format '{{.Image}}'`. Setelah merekam bukti, jalankan `docker rm -f cloudlab-api`.

Gambar 8. Bukti build dan respons API Lab 05 yang dijalankan kembali

```
docker build
docker run
```

```
Lab 05 - Docker API aktual

docker build cloud-notes-api:lab05 -> sukses

docker run -p 127.0.0.1:8000:8000 -> clouddlab-api aktif

GET /health -> HTTP 200 {'status': 'ok', 'service': 'cloud-notes-api'}

GET /runtime -> HTTP 200 {'environment': 'classroom', 'storage': 'memory'}

POST /notes -> HTTP 201 id=1 title=Docker

GET /notes -> HTTP 200 count=1

POST /notes title="" " -> HTTP 422

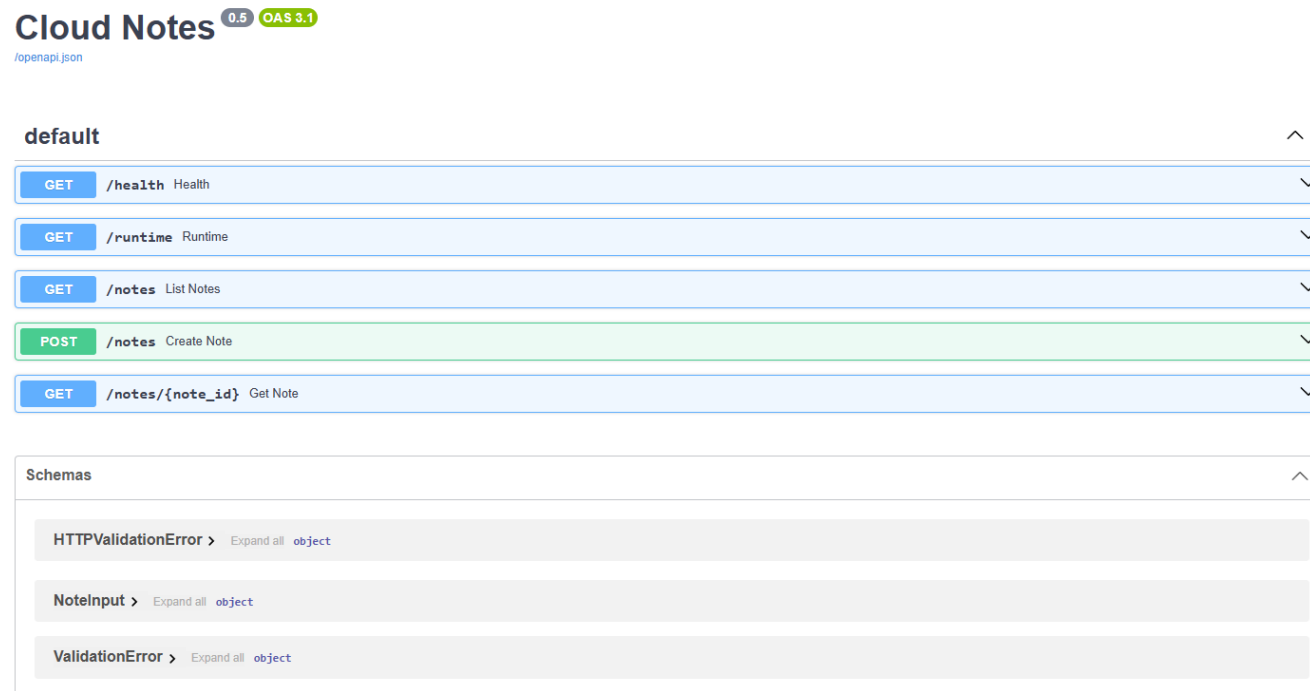
GET /notes/9999999 -> HTTP 404
```

Output HTTP aktual ditata ulang agar terbaca. Password/token tidak direkam.

Perintah: `docker build`, `docker run`, GET health/runtime, POST dan GET notes, lalu uji judul spasi. Cuplikan hasil Docker dan HTTP aktual ditata agar terbaca. HTTP 201 berarti catatan dibuat; HTTP 422 menunjukkan validasi; container baru mengembalikan daftar kosong karena data hanya di memori.

Gambar 9. Dokumentasi OpenAPI dari container Lab 05 yang berjalan

```
http://127.0.0.1:8000/docs
docker run
/health
/runtime
/notes
```



Perintah: buka <http://127.0.0.1:8000/docs> setelah `docker run`. Browser mengambil Swagger UI dari API yang sedang hidup. Gunakan route `/health`, `/runtime`, dan `/notes` untuk membaca kontrak sebelum memanggil endpoint.

Gambar 10. Tantangan stats yang diuji pada image solusi sementara

```
/stats  
stats-proof
```

```
Lab 05 - solusi /stats, image dibuild dari salinan kunci  
GET /stats sebelum POST -> HTTP 200, 0 catatan, 0 karakter  
POST /notes (Ops) -> HTTP 201  
POST /notes (Incident 42) -> HTTP 201  
GET /stats -> HTTP 200, 2 catatan, judul terpanjang 11  
POST judul berisi spasi saja -> HTTP 422
```

Hasil Docker/API aktual ditata ulang agar terbaca; kode starter tetap tanpa endpoint tantangan.

Perintah: tambah kode `/stats` di salinan uji, build image `stats-proof`, lalu GET sebelum/sesudah dua POST. Gambar adalah hasil API aktual yang ditata agar terbaca: jumlah 0→2, judul terpanjang 11 karakter, dan input kosong HTTP 422. Starter repo tetap menyisakan tantangan untuk dikerjakan sendiri.

4. Kunci lima pertanyaan analisis

1. `RUN` dan `COPY` mengubah image saat build; `CMD` dijalankan saat container start. `ENTRYPOINT` dapat menetapkan executable utama yang menerima argumen dari `CMD`; lab ini cukup dengan `CMD Uvicorn`.
2. `.dockerignore` mencegah `.env` masuk build context sehingga konfigurasi lokal tidak ikut image. File rahasia tetap harus diabaikan Git.
3. Pada `127.0.0.1:8000:8000`, bagian kiri adalah alamat dan port laptop; bagian kanan adalah port aplikasi dalam container. Loopback membatasi akses dari host lain.
4. `Field(min_length=1)` menolak string kosong. Validator tambahan `reject_blank_text` memangkas spasi dan menolak teks yang hanya berisi spasi; FastAPI memberi 422 karena payload gagal validasi.
5. `_notes` hidup di memori proses. Mengganti container membuat memori baru dan daftar kosong. Lab 06 memindahkan sumber data ke PostgreSQL dalam named volume.

Pemeriksaan image dengan Docker Scout, opsional 5-10 menit

Kasus kerja harian: tim akan merilis API catatan. Build dan respons HTTP berhasil, tetapi tim masih perlu mengetahui paket apa yang ada dalam image, advisory yang terkait, serta pilihan pembaruan. Docker Scout menyusun atau membaca SBOM-daftar komponen software dan versinya-kemudian mencocokkannya dengan data kerentanan. Ini pelengkap pengujian fungsi, bukan pengganti tes API. [Konsep Scout](#)

Perumpamaan: resep adalah Dockerfile, paket bahan software adalah image, daftar bahan dan nomor batch adalah SBOM, dan Scout mencocokkannya dengan daftar recall. CVE bukan tanggal kedaluwarsa fisik; hasil scan harus dibaca bersama versi dan konteks penggunaan.

1. Siapkan lingkungan yang sesuai

Jalur kelas memakai **terminal Linux Codespaces** dan image `cloud-notes-api:theory-online`, yang dibangun oleh `bash scripts/demo-online.sh build`. Laptop tetap cukup browser. Docker Desktop sudah menyertakan Scout; Docker Engine di cloud belum tentu membawa plugin tersebut. Akun/login Docker dan koneksi ke layanan Scout disiapkan sebelum presentasi. Jangan menampilkan password, token, device code aktif, atau isi konfigurasi Docker. [Instalasi Scout, quickstart](#)

```
bash scripts/scout-demo.sh install
bash scripts/scout-demo.sh check
```

Script memasang binary **standalone Scout 1.26.0** secara eksplisit dan memverifikasi checksum di Linux; instalasinya berada pada cache pengguna khusus demo. Jika instalasi valid sudah ada, binary tersebut diperiksa dan dipakai ulang. Helper memakai binary ini, bukan plugin Docker CLI lain yang mungkin sudah terpasang. `check` memeriksa binary, daemon, dan image lokal, **bukan bukti login atau akses layanan**. Script tidak melakukan login, push, pengaktifan repository monitoring, atau perubahan container. Waktu unduhan dan login tidak termasuk slot demo 5-10 menit.

Jika Anda menggunakan Docker Desktop lokal: setelah image `cloud-notes-api:lab05` dibangun, gunakan `docker scout version`, lalu command CLI langsung pada contoh di bawah dengan target `local://cloud-notes-api:lab05`. Script installer kelas ditujukan untuk Linux, bukan PowerShell Windows. [Referensi Scout CLI](#)

2. Baca ringkasan, lalu satu temuan nyata

```
bash scripts/scout-demo.sh quickview
bash scripts/scout-demo.sh cves
bash scripts/scout-demo.sh recommendations
```

Command	Tujuan	Cara membaca
<code>quickview</code>	Ringkasan kerentanan image dan base image	Pastikan target benar; baca C/H/M/L/U sebagai critical/high/medium/low/unspecified. Angka bergantung image dan advisory saat scan.
<code>cves</code>	Rincian temuan menurut paket	Pilih satu temuan: nama paket, versi terpasang, CVE, severity, affected range, dan fixed version bila tercatat.
<code>recommendations</code>	Kandidat pembaruan base image	Baca saran dan dampaknya. Ini belum mengubah Dockerfile/image; tidak tersedia saran juga merupakan hasil yang mungkin.

Pada Docker Desktop lokal, padanan CLI:

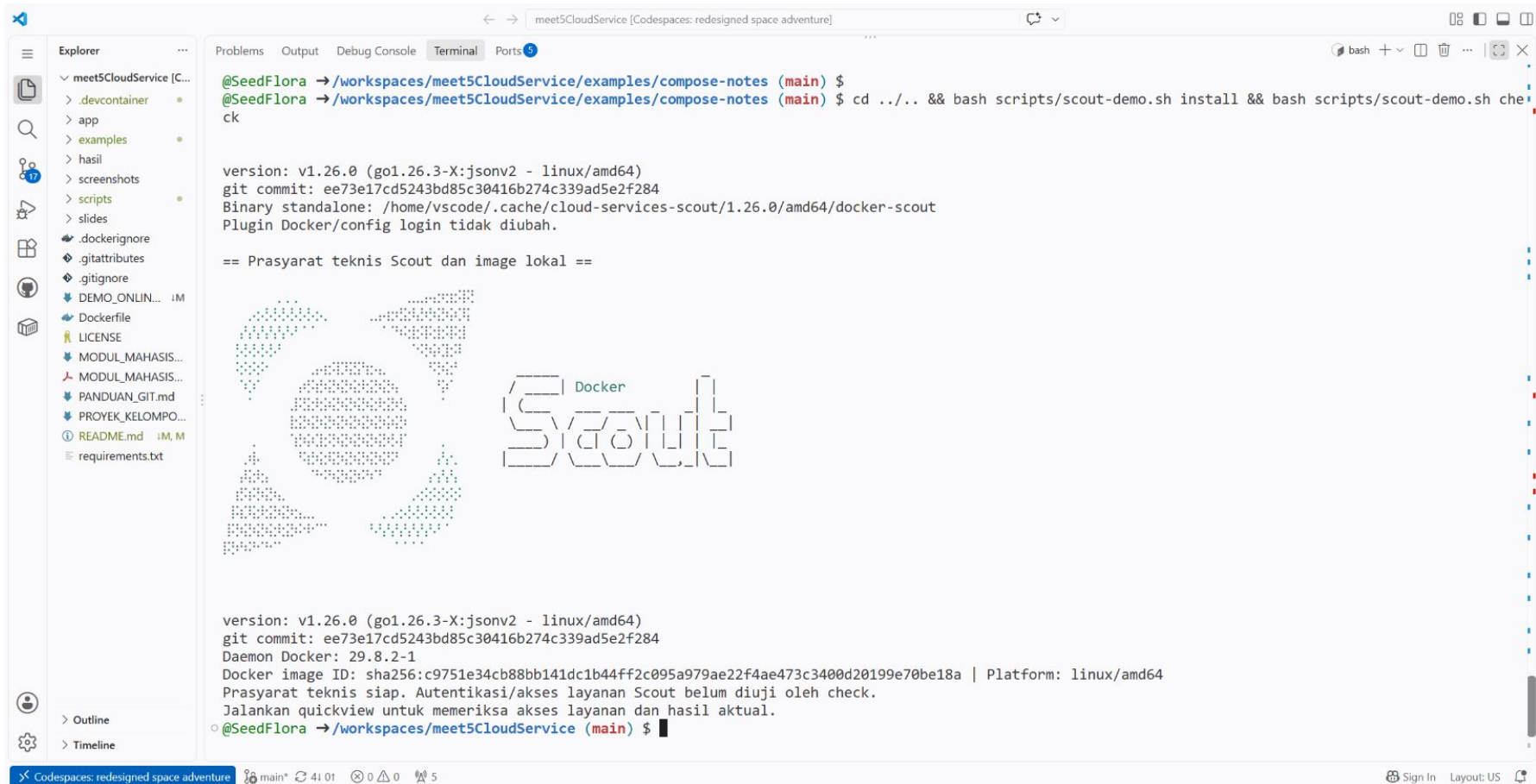
```
docker scout quickview local://cloud-notes-api:lab05
docker scout cves local://cloud-notes-api:lab05
docker scout recommendations local://cloud-notes-api:lab05
```

Target `local://` menggunakan image store lokal tanpa fallback mencari target di registry; kegiatan Scout tetap dapat membutuhkan jaringan dan autentikasi. Pada plugin Desktop, `docker scout cves --details local://cloud-notes-api:lab05` memberi rincian tambahan; jangan menambahkan flag tersebut pada helper yang hanya menerima satu nama action. [Quickview](#), [CVEs](#), [recommendations](#)

Rekaman baseline: screenshot Scout berikut merekam image sebelum pembaruan dependency kelas, digest `c9751e34cb88`. Tag dapat digunakan ulang; digest mewakili isi image. Source repo terbaru dibuild dan dianalisis sesuai hasil aktual, sehingga total peserta dapat berbeda. Baseline membantu memahami alasan perbaikan dan perbandingan sesudahnya.

Gambar 11. Instalasi Scout dan check teknis pada mesin Codespaces

```
bash scripts/scout-demo.sh install
bash scripts/scout-demo.sh check
```



```
@SeedFlora → /workspaces/meet5CloudService/examples/compose-notes (main) $
@SeedFlora → /workspaces/meet5CloudService/examples/compose-notes (main) $ cd ../../ && bash scripts/scout-demo.sh install && bash scripts/scout-demo.sh check

version: v1.26.0 (go1.26.3-X:jsonv2 - linux/amd64)
git commit: ee73e17cd5243bd85c30416b274c339ad5e2f284
Binary standalone: /home/vscode/.cache/cloud-services-scout/1.26.0/amd64/docker-scout
Plugin Docker/config login tidak diubah.

== Prasyarat teknis Scout dan image lokal ==

          Docker
        Scout

version: v1.26.0 (go1.26.3-X:jsonv2 - linux/amd64)
git commit: ee73e17cd5243bd85c30416b274c339ad5e2f284
Daemon Docker: 29.8.2-1
Docker image ID: sha256:c9751e34cb88bb141dc1b44ff2c095a979ae22f4ae473c3400d20199e70be18a | Platform: linux/amd64
Prasyarat teknis siap. Autentikasi/akses layanan Scout belum diuji oleh check.
Jalankan quickview untuk memeriksa akses layanan dan hasil aktual.
@SeedFlora → /workspaces/meet5CloudService (main) $
```

Command: `bash scripts/scout-demo.sh install`, lalu `bash scripts/scout-demo.sh check`, dari root repo. Fungsi: menyiapkan binary standalone v1.26.0 dan memeriksa daemon/image lokal. Baca hasil aktual: Linux/amd64, image ID berawalan c9751e34cb88, serta pesan akses layanan belum diuji. Check bukan scan CVE atau bukti autentikasi.

Gambar 12. Quickview berhasil menganalisis image API utama Lab 05

```
bash scripts/scout-demo.sh quickview
```

```
@SeedFlora → /workspaces/meet5CloudService (main) $ bash scripts/scout-demo.sh quickview
== Scout quickview: local://cloud-notes-api:theory-online ==
...Storing image for indexing
✓ Image stored for indexing
...Indexing
✓ Indexed 136 packages
✓ Provenance obtained from attestation
...Pulling
✓ Pulled
...Evaluating policies
✓ Policy evaluation completed
```

Target	digest	0C	4H	9M	28L
local://cloud-notes-api:theory-online	c9751e34cb88	0C	4H	9M	28L
Base image	python:3.12-slim	0C	1H	6M	27L

```
Policy status FAILED (2/7 policies met)
Health score E (22%)
```

Status	Policy	Results
✓	Default non-root user	
!	Copyleft licensed packages found	363 packages
!	Fixable critical or high vulnerabilities found	0C 3H 0M 0L
!	High-profile vulnerabilities found	0C 0H 1M 0L
!	Outdated base images found	
✓	No unapproved base images	0 deviations
!	Required supply chain attestations missing	1 deviation

```
What's next:
View policy violations → docker scout policy local://cloud-notes-api:theory-online
View vulnerabilities → docker scout cves local://cloud-notes-api:theory-online
Compare with the latest in the registry → docker scout compare --to-latest local://cloud-notes-api:theory-online

Selesai. Baca output actual pada /workspaces/meet5CloudService/hasil/scout/20261008T032806Z-gRzw0Y
@SeedFlora → /workspaces/meet5CloudService (main) $
```

Command: `bash scripts/scout-demo.sh quickview`. Fungsi: menghubungkan identitas image dengan ringkasan advisory. Snapshot 8 Oktober 2026 memakai `local://cloud-notes-api:theory-online`, digest `c9751e34cb88`, 136 paket diindeks, dan target `0C/4H/9M/28L`. Base `python:3.12-slim` tampil `0C/1H/6M/27L`. Angka bukan target kelulusan dan dapat berubah; tidak ada critical bukan berarti tidak ada risiko.

Output aktual masih menampilkan **Health score 22%** dan **Policy status FAILED**. Itu bukan persentase keamanan atau nilai praktikum. Fitur health scores dan sejumlah fitur dashboard tercantum pada [pengumuman retirement Docker](#); kelas membaca paket/CVE/fix dan keputusan teknis, bukan memakai skor tersebut sebagai syarat rilis. Keberhasilan scan pada Codespace uji ini tidak menjamin akses tanpa autentikasi pada lingkungan lain.

Gambar 13. Rincian CVE dan ringkasan 41 temuan pada 15 paket

```
bash scripts/scout-demo.sh cves
```

```
@SeedFlora → /workspaces/meet5CloudService (main) $ bash scripts/scout-demo.sh cves
Affected range : >=3.2.1-3
Fixed version  : not fixed

0C    0H    0M    1L  shadow 1:4.17.4-2
pkg:deb/debian/shadow@1%3A4.17.4-2?os_distro=trixie&os_name=debian&os_version=13

X LOW CVE-2007-5686
https://scout.docker.com/v/CVE-2007-5686?s=debian&n=shadow&ns=debian&t=deb&osn=debian&osv=13&vr=%3E0
Affected range : >0
Fixed version  : not fixed

0C    0H    0M    1L  apt 3.0.3
pkg:deb/debian/apt@3.0.3?os_distro=trixie&os_name=debian&os_version=13

X LOW CVE-2011-3374
https://scout.docker.com/v/CVE-2011-3374?s=debian&n=apt&ns=debian&t=deb&osn=debian&osv=13&vr=%3E0
Affected range : >0
Fixed version  : not fixed

41 vulnerabilities found in 15 packages
CRITICAL  0
HIGH      4
MEDIUM   9
LOW      28

What's next:
View base image update recommendations → docker scout recommendations local://cloud-notes-api:theory-online

Selesai. Baca output actual pada /workspaces/meet5CloudService/hasil/scout/20261008T032907Z-NDP541
@SeedFlora → /workspaces/meet5CloudService (main) $
```

Command: `bash scripts/scout-demo.sh cves`. Fungsi: membaca paket, versi, advisory, severity, affected range, dan fixed version. Pada cuplikan aktual: shadow 1:4.17.4-2 dengan LOW CVE-2007-5686, apt 3.0.3 dengan LOW CVE-2011-3374, keduanya not fixed. Ringkasan 0C/4H/9M/28L = 41 temuan pada 15 paket. Kunci: not fixed tidak boleh diganti dengan versi perbaikan tebakan; nilai konteks dan mitigasi, lalu pilih update yang terbukti sesuai. Potongan akhir ini tidak menampilkan empat high pada bagian lain laporan.

Gambar 14. Paket high memiliki jalur perbaikan dependency dan OS yang berbeda

```
grep -B 3 -A 8 'HIGH CVE' hasil/scout/*/cves.txt
```

```
@SeedFlora → /workspaces/meet5CloudService (main) $ grep -B 3 -A 8 'HIGH CVE' hasil/scout/*/cves.txt

Affected range : >=0.4.1
Fixed version : <1.3.1
CVSS Score : 7.5
CVSS Vector : CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

X HIGH CVE-2026-48818 [Server-Side Request Forgery (SSRF)]
https://scout.docker.com/v/CVE-2026-48818?s=github&n=starlette&t=pypi&vr=%3C1.1.0
Affected range : <1.1.0
Fixed version : 1.1.0
CVSS Score : 7.5
CVSS Vector : CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

X HIGH CVE-2025-62727 [Uncontrolled Resource Consumption]
https://scout.docker.com/v/CVE-2025-62727?s=github&n=starlette&t=pypi&vr=%3E%3D0.39.0%2C%3C%3D0.49.0
Affected range : >=0.39.0
Fixed version : <=0.49.0
CVSS Score : 7.5
CVSS Vector : CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

X MEDIUM CVE-2026-48710 CISA KEV [Improper Validation of Unsafe Equivalence in Input]
--
0C 1H 0M 0L zlib 1:1.3.dfsg+really1.3.1-1
pkg:deb/debian/zlib@1%3A1.3.dfsg%2Breally1.3.1-1?os_distro=trixie&os_name=debian&os_version=13

X HIGH CVE-2026-85091
https://scout.docker.com/v/CVE-2026-85091?s=debian&n=zlib&ns=debian&t=deb&osn=debian&osv=13&vr=%3E0
Affected range : >0
Fixed version : not fixed

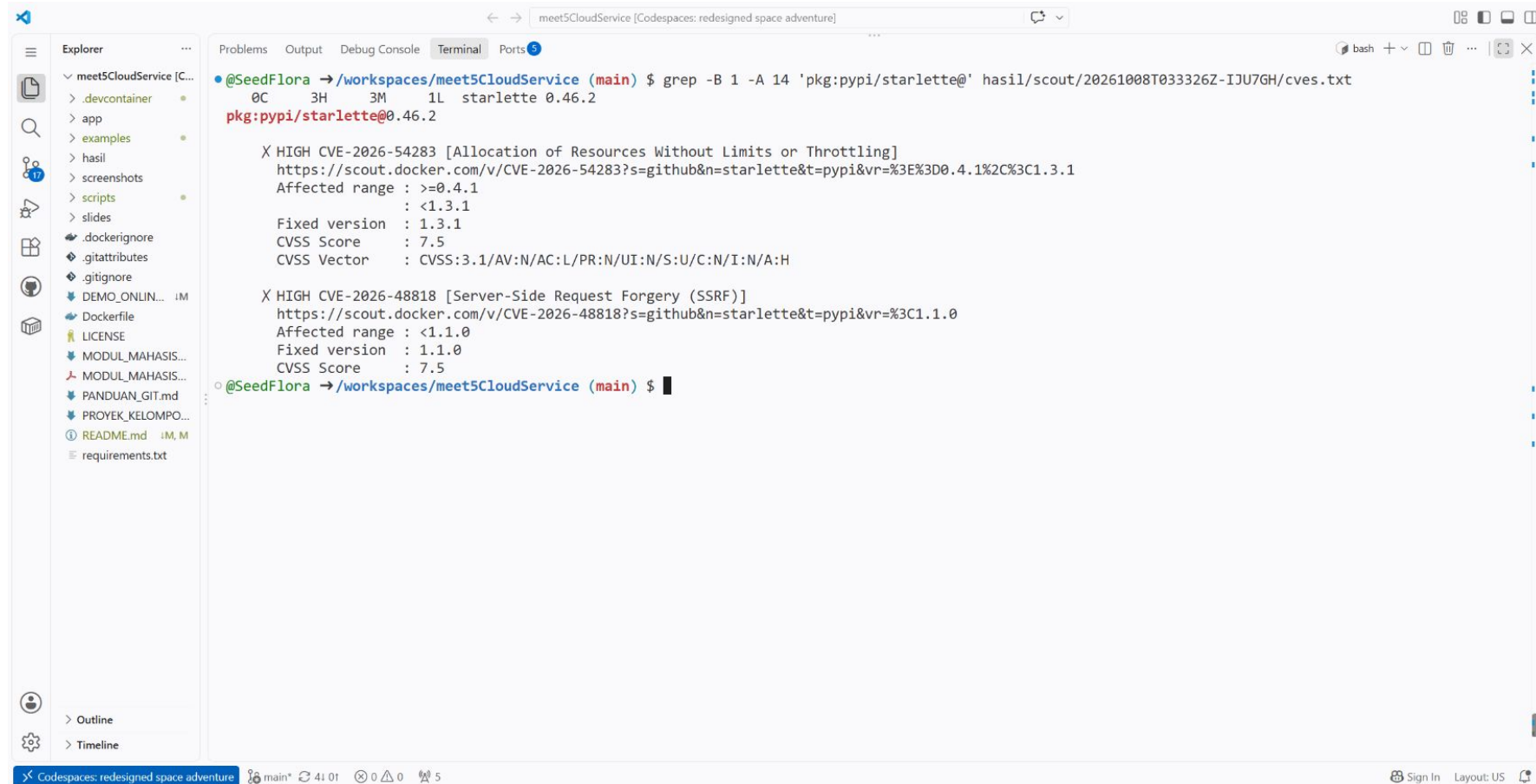
0C 0H 5M 1L pip 25.0.1
pkg:pypi/pip@25.0.1

@SeedFlora → /workspaces/meet5CloudService (main) $
```

Command rekaman: `grep -B 3 -A 8 'HIGH CVE' hasil/scout/*/cves.txt`, membaca hasil tersimpan. Bila ada banyak laporan, pilih file baseline yang sesuai metadata/digest agar tidak mencampur image. Baca output: temuan Starlette memiliki fixed version berbeda menurut advisory (1.3.1, 1.1.0, 0.49.1); zlib menampilkan high CVE-2026-85091/not fixed. Kunci: update dependency harus kompatibel dengan constraint FastAPI, lalu build image baru, tes API, dan rescans. Paket OS memerlukan review base/OS/mitigasi. Severity tidak membuktikan seluruh route aplikasi dapat dieksplorasi; periksa advisory dan konteks.

Gambar 15. Versi Starlette terpasang dibandingkan dengan rentang terdampak dan fixed version

```
grep -B 1 -A 14 'pkg:pypi/starlette@' hasil/scout/20261008T033326Z-IJU7GH/cves.txt
```



Command rekaman: `grep -B 1 -A 14 'pkg:pypi/starlette@' hasil/scout/20261008T033326Z-IJU7GH/cves.txt`, membaca satu laporan baseline eksplisit; path mesin peserta mengikuti hasil helper. Baca: installed 0.46.2, HIGH CVE-2026-54283 affected $\geq 0.4.1 < 1.3.1$ fixed 1.3.1; HIGH CVE-2026-48818 affected $< 1.1.0$ fixed 1.1.0. Kunci: versi terpasang berada dalam rentang itu; memilih 1.1.0 belum memenuhi fix advisory pertama. Evaluasi update FastAPI/dependency yang kompatibel, verifikasi versi dalam image, tes API, lalu rescan.

Kunci konteks: advisory pertama memerlukan parsing form urlencoded; yang kedua StaticFiles Windows dan mengecualikan POSIX. Contoh JSON/Linux tidak memakai fitur itu. Scan paket perlu dilengkapi tinjauan kode/config; update kompatibel tetap diuji. [Advisory form](#), [advisory Windows](#)

Gambar 16. Rekomendasi aktual: base sudah terbaru dan tidak ada kandidat tag lain

```
bash scripts/scout-demo.sh recommendations
```

```
@SeedFlora → /workspaces/meet5CloudService (main) $ bash scripts/scout-demo.sh recommendations
...Indexing
✓ Indexed 136 packages
✓ Provenance obtained from attestation
...Pulling
✓ Pulled

Target | local://cloud-notes-api:theory-online
digest | c9751e34cb88

## Recommended fixes

Base image is python:3.12-slim

Name | 3.12-slim
Digest | sha256:2b4f19dae3a777dfc3b76730bda1e82e1f66ab2a2686fa93ca78edbf4f04ffe
Vulnerabilities | 0C 1H 6M 27L
Pushed | 2 days ago
Size | 43 MB
Packages | 124
Runtime | 3.12.15

Refresh base image
Rebuild the image using a newer base image version. Updating this may result in breaking changes.

✓ This image version is up to date.

Change base image
The list displays new recommended tags in descending order, where the top results are rated as most suitable.

✓ There are no tag recommendations at this time.

Selesai. Baca output actual pada /workspaces/meet5CloudService/hasil/scout/20261008T033206Z-SvFg0p
@SeedFlora → /workspaces/meet5CloudService (main) $
```

Command: `bash scripts/scout-demo.sh recommendations`. Fungsi: meninjau opsi refresh/tag base tanpa mengubah aplikasi. Baca hasil baseline: python:3.12-slim dengan runtime 3.12.15, versi image up to date, no tag recommendations; base tetap 0C/1H/6M/27L. Kunci: tidak ada saran bukan berarti aman. Temuan dependency Starlette ditinjau melalui dependency/framework; jangan mengarang tag base baru atau mengklaim perubahan otomatis.

Gambar 17. Laporan baseline berhasil menulis CVEs dalam format SARIF

```
bash scripts/scout-demo.sh report
```

```

@SeedFlora → /workspaces/meet5CloudService (main) $ bash scripts/scout-demo.sh report
Base image is python:3.12-slim

Name          | 3.12-slim
Digest        | sha256:2b4f19dae3a777dfc3b76730bda1e82e1f66ab2a2686fa93ca78edbf4f04ffe
Vulnerabilities | 0C 1H 6M 27L
Pushed       | 2 days ago
Size         | 43 MB
Packages     | 124
Runtime      | 3.12.15

Refresh base image
Rebuild the image using a newer base image version. Updating this may result in breaking changes.

✓ This image version is up to date.

Change base image
The list displays new recommended tags in descending order, where the top results are rated as most suitable.

✓ There are no tag recommendations at this time.

== Scout cves: local://cloud-notes-api:theory-online ==
...Storing image for indexing
✓ Image stored for indexing
...Indexing
✓ Indexed 136 packages
✓ Provenance obtained from attestation
...Pulling
✓ Pulled
X Detected 15 vulnerable packages with a total of 41 vulnerabilities
✓ Report written to /workspaces/meet5CloudService/hasil/scout/20261008T033326Z-IJU7GH/cves.sarif.json

Selesai. Baca output actual pada /workspaces/meet5CloudService/hasil/scout/20261008T033326Z-IJU7GH
@SeedFlora → /workspaces/meet5CloudService (main) $

```

Command: `bash scripts/scout-demo.sh report`. Fungsi: menyimpan ringkasan, rincian, rekomendasi, dan SARIF secara berurutan. Baca hasil aktual: 15 vulnerable packages/41 findings dan Report written to `.../cves.sarif.json`, lalu selesai. Kunci: SARIF adalah format data temuan; tersimpan bukan berarti paket diperbaiki. Metadata/output/exit code generated diabaikan Git, dapat dipakai review pribadi, tanpa kewajiban laporan lab baru.

3. Kunci diskusi dan keputusan engineer

1. **Build dan HTTP 200 sudah berhasil; mengapa scan?** Itu membuktikan fungsi dasar aplikasi. Scan memeriksa komposisi software terhadap advisory yang diketahui; masalah dependency belum tentu muncul pada request pengujian.
2. **Ada severity high; langsung ganti semua versi?** Tentukan asal paket dan versi terdampaknya terlebih dahulu. Temuan dari OS/base biasanya ditangani melalui base image; temuan dependency aplikasi melalui requirements atau lockfile. Periksa ketersediaan fix, penggunaan fungsi rentan, akses jaringan, konfigurasi, dan kompatibilitas. Severity membantu prioritas, tetapi bukan satu-satunya dasar keputusan.
3. **Apa arti fixed version kosong atau not fixed?** Belum ada versi perbaikan yang dicatat pada data tersebut. Temuan tetap perlu dinilai; mitigasi konfigurasi, pembatasan akses, pembaruan lain, atau penggantian komponen dapat relevan. Jangan menghapus temuan agar angka menjadi nol.
4. **Filter critical/high kosong; apakah image bebas kerentanan?** Tidak ada temuan yang memenuhi filter pada cakupan dan waktu scan tersebut. Baca laporan lengkap; jangan menyamakan filter kosong dengan scan tanpa temuan atau jaminan keamanan.
5. **recommendations memberi base baru; apa langkah berikutnya?** Periksa bahwa runtime/platform/dependency kompatibel, ubah **FROM** bila sesuai, rebuild image, buat container baru, jalankan health dan alur POST/GET/validasi, lalu scan ulang. Rebuild tidak otomatis mengganti container lama.
6. **Bagaimana jika total CVE menjadi 0?** Artinya tidak ada kerentanan yang terdeteksi oleh data dan cakupan scan itu. Tetap periksa fungsi, secret, konfigurasi, izin, exposure jaringan, dan risiko yang belum dikenal; 0 bukan sertifikat aman.
7. **Plugin, login, atau backend gagal; apakah hasilnya 0?** Belum ada hasil scan yang valid. Selesaikan prasyarat atau gunakan rekaman aktual yang berlabel image/digest dan waktu. Kegagalan layanan Scout berbeda dari kegagalan API.

Setiap action scan menyimpan output dan exit code pada `hasil/scout/<waktu-UTC>-<acak>/`. Untuk rangkaian laporan lengkap, jalankan `bash scripts/scout-demo.sh report`. Rangkaian ini mencoba `quickview.txt`, `cves.txt`, `recommendations.txt`, lalu `cves.sarif.json` dan `cves-sarif.console.txt`; `metadata.txt` mencatat target dan status. Proses berhenti pada kegagalan pertama, jadi folder dapat berisi laporan parsial. Folder generated diabaikan Git. Laporan parsial atau gagal tidak boleh diberi label lulus; exit 0 sendiri juga tidak menyatakan bebas kerentanan. Tidak ada kewajiban mengumpulkan laporan lab baru.

Analisis Scout

4. Kunci perbaikan dan bukti sesudah update

Source API utama telah diperbarui ke **FastAPI 0.142.4/Uvicorn 0.54.0**; build kelas resolve **Starlette 1.7.0**. Verifikasi versi dalam image, karena pin dependency langsung tidak mengunci seluruh paket transitif. Regresi lokal API + solusi stats meluluskan **21/21 pemeriksaan**. API tetap JSON/Linux, UID 10001, dan memori; perbaikan dependency tidak menjadikannya database.

Dari root repo pada terminal Linux, setelah memakai source terbaru:

Command yang dapat disalin

```
bash scripts/demo-online.sh stop
bash scripts/demo-online.sh start
bash scripts/demo-online.sh test --restart
docker exec cloudlab-theory-online python -c \
    'import fastapi, starlette, uvicorn; print("VERSIONS", fastapi.__version__, starlette.__version__, uvicorn.__version__)'
docker tag cloud-notes-api:theory-online cloud-notes-api:scout-improved
SCOUT_IMAGE=cloud-notes-api:scout-improved bash scripts/scout-demo.sh quickview
SCOUT_IMAGE=cloud-notes-api:scout-improved bash scripts/scout-demo.sh cves
```

Kunci command: stop/start membuat container demo berlabel dari source baru; catatan memori hilang. `test --restart` membuktikan fungsi HTTP serta perilaku reset; versi dibaca melalui Python dalam container. `docker tag` menambahkan nama lokal image yang sama, tidak upload. `SCOUT_IMAGE` memilih target scan; default helper tetap theory-online. Snapshot memakai host 8000; port alternatif pada wrapper harus konsisten. Pengajar menyimpan baseline sebagai scout-before sebelum rebuild; peserta tidak perlu menurunkan dependency untuk meniru baseline.

Gambar 18. HTTP API setelah update berhasil dan versi dalam container diperiksa

test --restart, docker exec pemeriksaan versi, docker tag kandidat.

```
@SeedFlora → /workspaces/meet5CloudService (main) $
@SeedFlora → /workspaces/meet5CloudService (main) $ bash scripts/demo-online.sh test --restart && docker exec cloudlab-theory-online python -c 'import fast
otes-api:scout-improved
== HTTP: layanan hidup, konfigurasi runtime, validasi, data, dokumentasi ==
PASS GET /health → HTTP 200
{"status": "ok", "service": "cloud-notes-api"}
PASS GET /runtime → HTTP 200
{"environment": "classroom", "storage": "memory"}
PASS POST /notes → HTTP 201
{"id": 1, "title": "Demo Docker teori", "content": "Resep → image → container; data sementara di memori."}
PASS GET /notes/1 → HTTP 200
{"id": 1, "title": "Demo Docker teori", "content": "Resep → image → container; data sementara di memori."}
PASS GET /notes → HTTP 200
[{"id": 1, "title": "Demo Docker teori", "content": "Resep → image → container; data sementara di memori."}]
PASS POST /notes → HTTP 422
{"detail": [{"type": "value_error", "loc": ["body", "title"], "msg": "Value error, teks tidak boleh hanya berisi spasi", "input": " ", "ctx": {"error": {}}}]
PASS GET /notes/0 → HTTP 404
{"detail": "Note tidak ditemukan"}
PASS GET /docs → HTTP 200

== Restart proses: catatan in-memory menghilang (bukan penyimpanan database) ==
cloudlab-theory-online
PASS GET /notes → HTTP 200
[]
PASS: data sementara hilang sesudah restart. Volume saja tidak mengubah dict Python menjadi database.

Semua pemeriksaan demo lulus. Ini demonstrasi formatif, tanpa tugas atau nilai lab terpisah.
VERSIONS 0.142.4 1.7.0 0.54.0
@SeedFlora → /workspaces/meet5CloudService (main) $
```

Command: test --restart, docker exec pemeriksaan versi, docker tag kandidat. Fungsi: membuktikan perubahan tetap menjalankan health/runtime/create/read/validasi/missing/docs dan memori hilang setelah restart. Baca hasil aktual: PASS, daftar [], VERSIONS 0.142.4/1.7.0/0.54.0 dalam urutan FastAPI/Starlette/Uvicorn. Field API utama environment/storage memory berbeda dari app_env/storage postgres pada Compose.

Gambar 19. Scan ulang tag hasil perbaikan menampilkan 0C 1H 6M 27L

```
SCOUT_IMAGE=cloud-notes-api:scout-improved bash scripts/scout-demo.sh quickview
```

```
@SeedFlora → /workspaces/meet5CloudService (main) $ SCOUT_IMAGE=cloud-notes-api:scout-improved bash scripts/scout-demo.sh quickview
== Scout quickview: local://cloud-notes-api:scout-improved ==
...Storing image for indexing
✓ Image stored for indexing
...Indexing
✓ Indexed 138 packages
✓ Provenance obtained from attestation
...Pulling
✓ Pulled
...Evaluating policies
✓ Policy evaluation completed

Target      local://cloud-notes-api:scout-improved    0C    1H    6M    27L
digest      39d3c08210c2
Base image   python:3.12-slim                        0C    1H    6M    27L

Policy status FAILED (4/7 policies met)
Health score C (67%)

Status | Policy | Results
--- | --- | ---
✓ | Default non-root user | 363 packages
! | Copyleft licensed packages found | 0C 0H 0M 0L
✓ | No fixable critical or high vulnerabilities | 0C 0H 0M 0L
✓ | No high-profile vulnerabilities | 0C 0H 0M 0L
! | Outdated base images found | 0 deviations
✓ | No unapproved base images | 1 deviation
! | Required supply chain attestations missing |

What's next:
View policy violations → docker scout policy local://cloud-notes-api:scout-improved
View vulnerabilities → docker scout cves local://cloud-notes-api:scout-improved
Compare with the latest in the registry → docker scout compare --to-latest local://cloud-notes-api:scout-improved

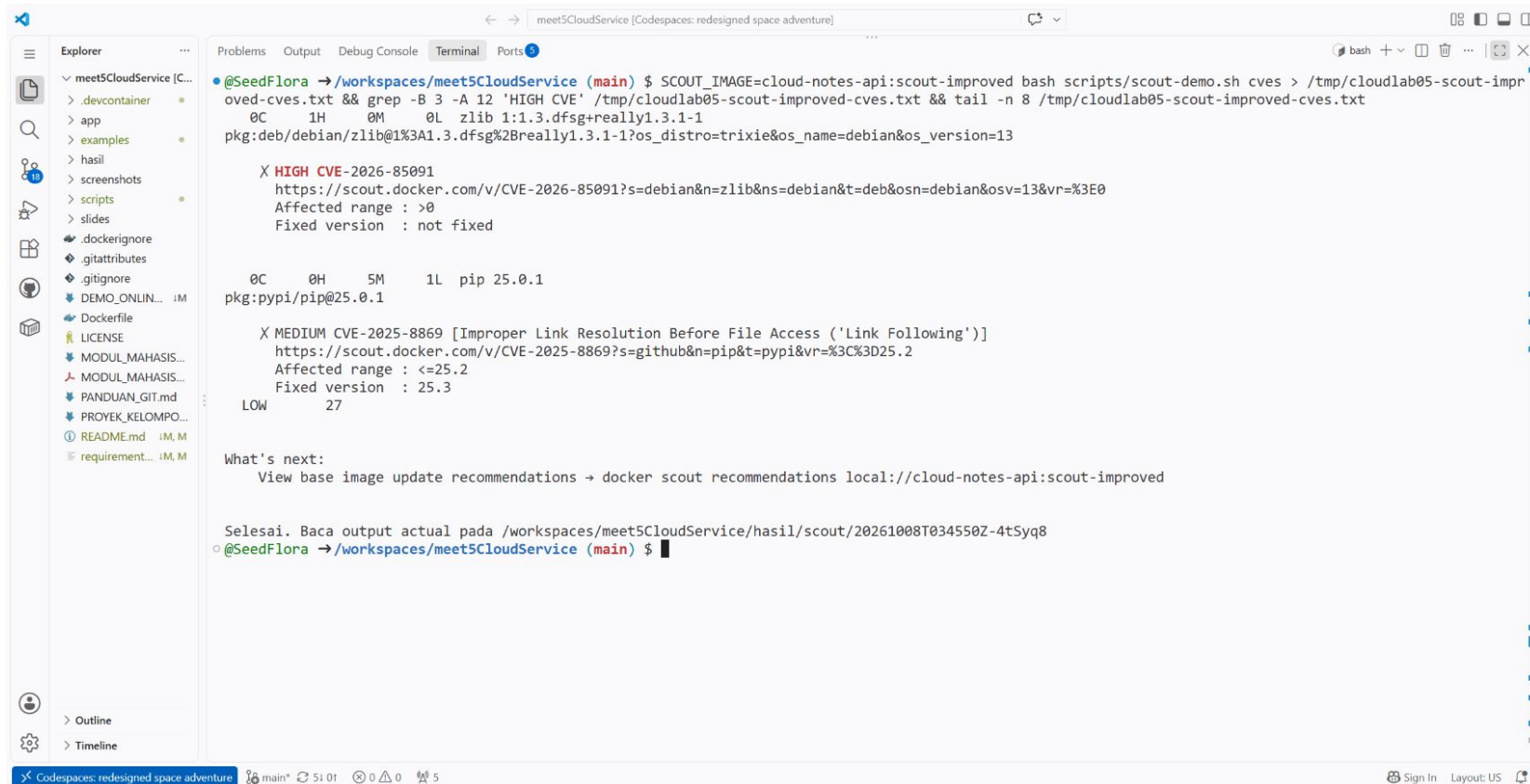
Selesai. Baca output actual pada /workspaces/meet5CloudService/hasil/scout/20261008T034506Z-eczYoI
@SeedFlora → /workspaces/meet5CloudService (main) $
```

Command: `SCOUT_IMAGE=cloud-notes-api:scout-improved bash scripts/scout-demo.sh quickview`. Fungsi: memeriksa artefak baru, digest 39d3c08210c2. Hasil aktual: 138 paket diindeks, target/base 0C/1H/6M/27L; baseline digest c9751e34cb88 berjumlah 136 paket dengan 0C/4H/9M/28L. Angka ini snapshot 8 Oktober 2026. Legacy health 67% bukan persentase keamanan/nilai kelas.

Kunci keputusan akhir: pembaruan menurunkan temuan dan tes fungsi tetap lulus, tetapi satu high masih perlu review advisory/fix/konteks/mitigasi. Jumlah paket yang naik tidak otomatis menaikkan risiko; baca paket/versi dan hasil rinci. Tetap periksa konfigurasi, secret, akses, dan risiko yang belum terdeteksi. [Alur lengkap beserta perbandingan](#) menjelaskan versi, tag, dan batas bukti; bukan tugas tambahan.

Gambar 20. Satu high tersisa adalah zlib OS dengan not fixed

```
SCOUT_IMAGE=cloud-notes-api:scout-improved bash scripts/scout-demo.sh cves > /tmp/cloudlab05-scout-improved-cves.txt
grep -B 3 -A 12 'HIGH CVE' ...
tail -n 8 ...
```



Command: `SCOUT_IMAGE=cloud-notes-api:scout-improved bash scripts/scout-demo.sh cves > /tmp/cloudlab05-scout-improved-cves.txt`, dilanjutkan `grep -B 3 -A 12 'HIGH CVE' ...` dan `tail -n 8 ...` dengan `&&` seperti blok lengkap pada [panduan online](#). Fungsi: baca rincian residual dari scan target yang benar. Hasil: zlib Debian 1:1.3.dfsg+really1.3.1-1, HIGH CVE-2026-85091, affected >0, not fixed. Tiga high Starlette baseline telah berkurang; dependency Python update tidak memperbaiki OS otomatis.

Kunci residual risk: catat digest/tanggal/advisory serta konteks penggunaan, review catatan distro/upstream dan mitigasi, lalu rencanakan follow-up. Versi Debian mempunyai epoch/revisi; jangan membandingkan string itu secara naif dengan versi upstream atau menebak versi fix. [Tracker Debian](#) masih mencatat unfixed pada tinjauan ini. Hasil 0C/1H/6M/27L serta tes fungsi PASS tetap perlu keputusan risiko yang sesuai untuk rilis kerja; demo kelas menggunakan data dummy.

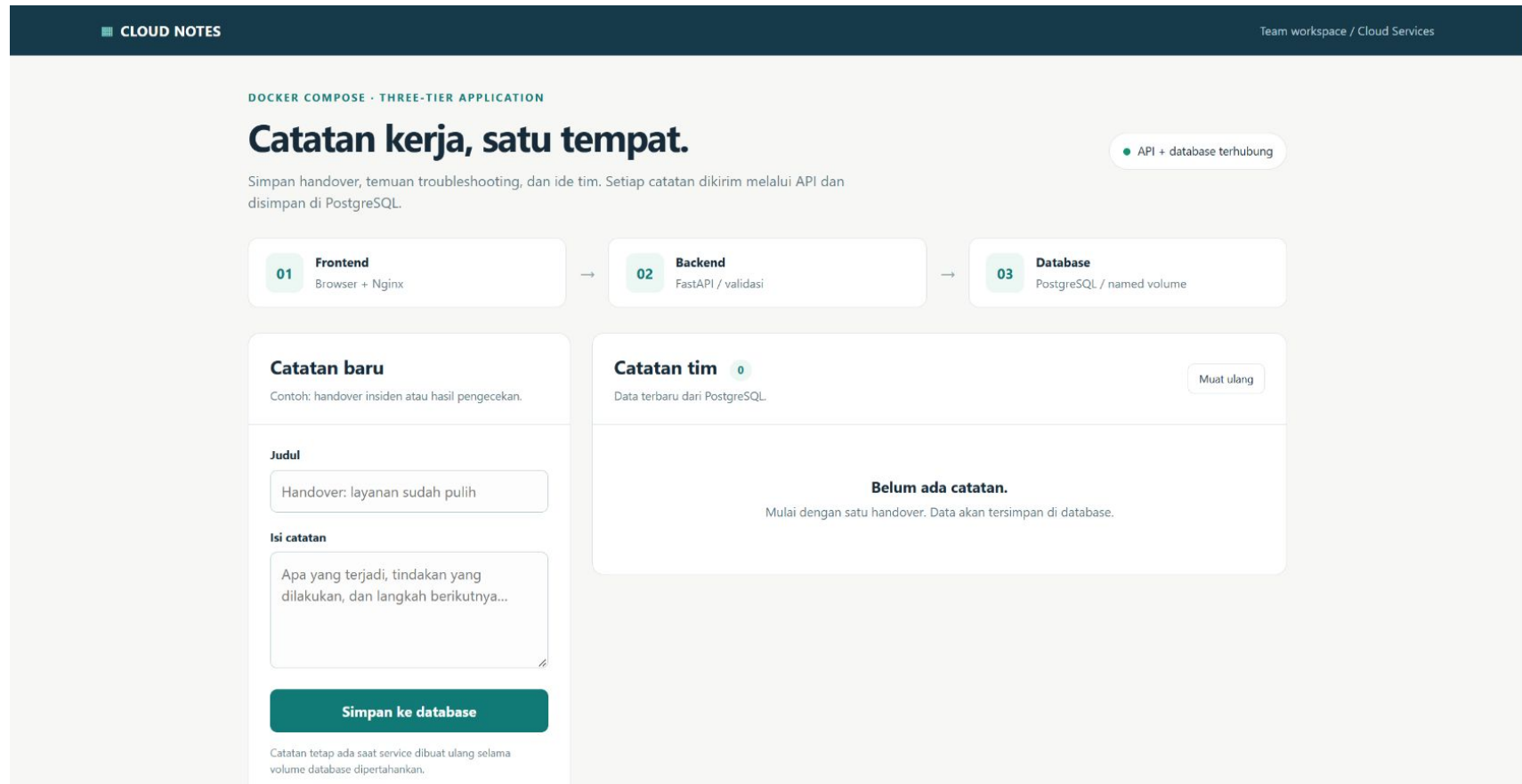
Jembatan ke aplikasi tiga layanan

Ikuti [demo Compose lengkap](#) untuk melihat frontend mengirim request ke FastAPI dan FastAPI menyimpan catatan pada PostgreSQL. Panduan mencakup command config/build/run/status/logs, web, pengujian HTTP, pembuktian persistensi, cleanup yang mempertahankan volume, serta kunci semua pertanyaan diskusi. Ini contoh terpisah menuju Lab 06; starter API memori dan kunci `/stats` di atas tetap menggunakan perilaku memori.

Setelah tiga service sehat, jalankan `bash scripts/smoke.sh` dari `examples/compose-notes`. Python pemeriksa berjalan di container API dan menguji URL internal web/proxy. Sembilan pemeriksaan mencakup GET 200, POST 201, judul kosong 422, ID tidak ditemukan 404, serta DELETE 204 untuk catatan sementara yang baru dibuat script. Kunci: 422/404 dapat menjadi PASS ketika sengaja diharapkan; `finally` mencoba membersihkan hanya ID hasil POST sendiri. Catatan dosen tetap tersedia untuk bukti persistensi. [Bagian smoke script](#) memuat tabel lengkap, cara membaca error, dan screenshot aktual 9/9 PASS; ini latihan bersama, bukan tugas tambahan.

Gambar 21. Frontend tiga layanan pada demo Compose online

```
examples/compose-notes  
docker compose up -d --build --wait  
/api/*
```



Langkah: dari `examples/compose-notes`, jalankan `docker compose up -d --build --wait`, lalu buka frontend melalui URL Ports. Default web 13006/API 13005; screenshot online memakai web 13106/API 13105. Frontend relatif `/api/*` diteruskan Nginx ke backend dan PostgreSQL; tanda terhubung harus dilengkapi tes HTTP serta simpan/baca.

Gambar 22. Catatan handover berhasil dibuat dari frontend Compose

isi judul/isi, lalu klik Simpan ke database

CLOUD NOTES

Team workspace / Cloud Services

DOCKER COMPOSE · THREE-TIER APPLICATION

Catatan kerja, satu tempat.

Simpan handover, temuan troubleshooting, dan ide tim. Setiap catatan dikirim melalui API dan disimpan di PostgreSQL.

● API + database terhubung

01 Frontend
Browser + Nginx

→

02 Backend
FastAPI / validasi

→

03 Database
PostgreSQL / named volume

Catatan baru

Contoh: handover insiden atau hasil pengecekan.

Judul

Handover: layanan sudah pulih

Isi catatan

Apa yang terjadi, tindakan yang dilakukan, dan langkah berikutnya...

Simpan ke database

Catatan tetap ada saat service dibuat ulang selama volume database dipertahankan.

Catatan tim 1

Data terbaru dari PostgreSQL.

Muat ulang

Handover: layanan checkout pulih

Tim mengganti konfigurasi koneksi database, memeriksa /health, lalu mencatat langkah pemulihan untuk shift berikutnya. Data ini contoh untuk kelas.

Hapus

#1 · 8/10/2026, 10.21.58

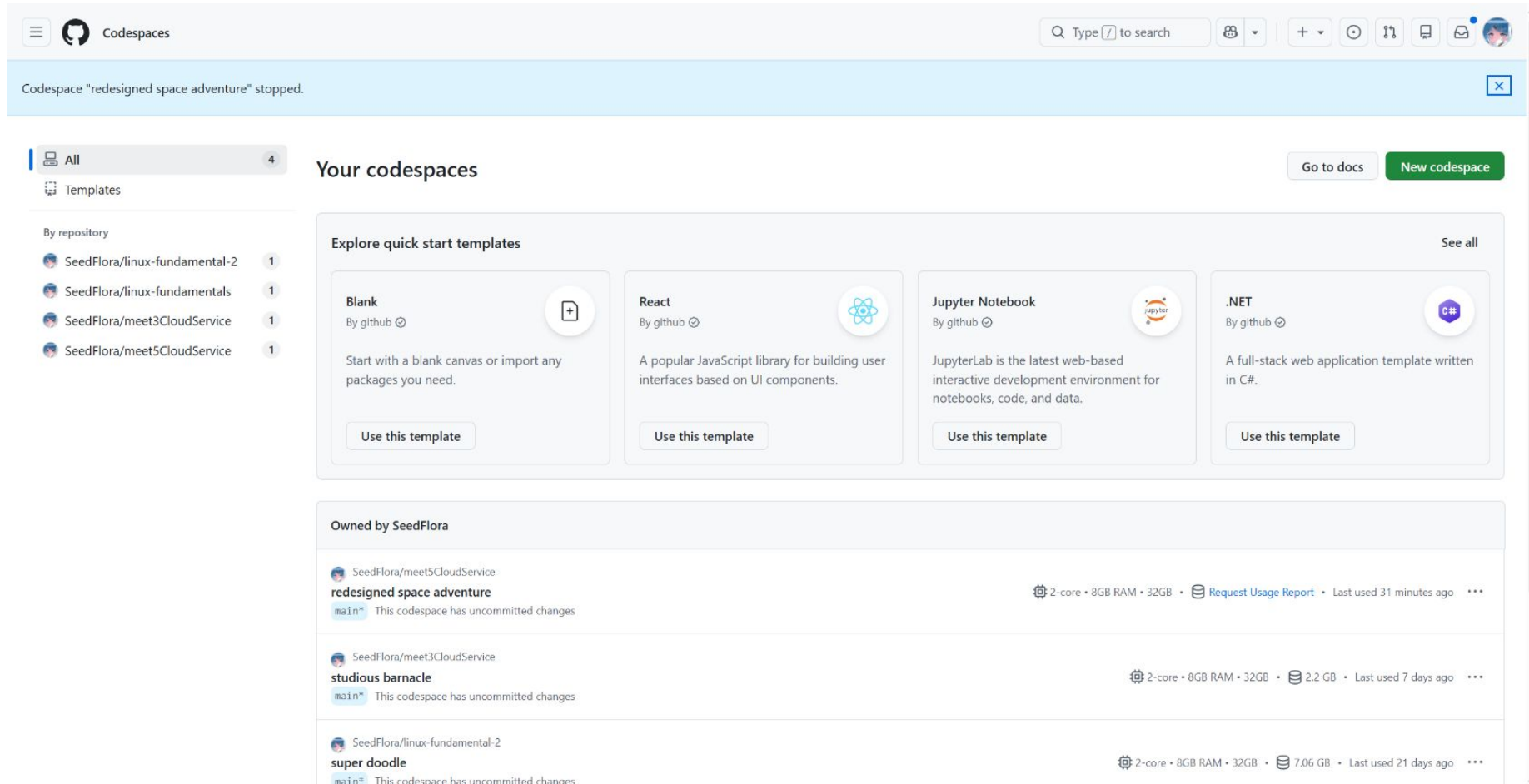
Langkah: isi judul/isi, lalu klik Simpan ke database. Screenshot aktual memakai data dummy Handover: layanan checkout pulih. Backend menyimpan melalui POST lalu frontend membaca daftar kembali. Panduan Compose melanjutkan ke pembuktian SQL/persistensi dan cleanup, bukan menganggap tampilan UI sebagai satu-satunya bukti.

Tutup lingkungan online setelah semua percobaan

Dari folder `examples/compose-notes`, `docker compose down` menghentikan project tanpa menghapus named volume. Kembali ke root dengan `cd ../../`, lalu `bash scripts/demo-online.sh stop` membersihkan container memori milik demo. Sesudah itu pilih **Stop codespace** pada halaman Codespaces; ini menghentikan compute. Menutup tab atau hanya down container belum menghentikan mesin cloud.

Gambar 23. Codespace kelas stopped setelah alur Compose dan Scout

github.com/codespaces → menu ... lingkungan kelas → Stop codespace.



Langkah: github.com/codespaces → menu ... lingkungan kelas → Stop codespace. Fungsi: menghentikan compute. Bukti aktual: banner Codespace redesigned space adventure stopped. Storage tetap dihitung selama disimpan; Stop menyimpan lingkungan dan perubahan lokal. Sebelum Delete, simpan/push kode yang perlu dipertahankan dan backup data penting, karena Delete menghilangkan data lokal. Generated reports/.env tidak ikut dipush secara otomatis.